gitit

A natural-language interface for Git

> **Say what you mean. See what will happen. Stay in control.** git + get it

The problem

Git is foundational to software development, but its interface asks people to recall a large, irregular command vocabulary precisely when they are trying to focus on their work. Even experienced developers pause to look up branch syntax, remember the order of rebase arguments, or compare reset, restore, and revert. Each interruption is small; the accumulated friction, hesitation, and risk are not.

The product

gitit is an interactive command-line tool that turns plain-language intent into an accurate Git workflow. Users describe the outcome they want; gitit returns the executable command immediately, followed by a one-sentence explanation. State-changing commands never run without confirmation.

```
$ gitit undo my last commit but keep all my changes
1. git reset --soft HEAD~1 - removes the commit; keeps changes staged
2. git reset HEAD~1 - removes the commit; keeps changes unstaged
Choose [1-2]  [e] Edit  [c] Copy  [q] Cancel
```

Example requests: `switch to main branch` | `rebase current branch` | `pull from main remote` | `show me diff b/w branch a and main` | `create a pull request`. When several operations are plausible, gitit ranks candidates, explains their differences, and lets the user choose, edit, copy, or cancel.

Experience principles

1 Intent first	State the desired outcome in ordinary language.
2 Command visible	Always see the exact executable and arguments.
3 Meaning explained	Get a concise, outcome-oriented explanation.
4 Risk proportional	Receive stronger checks for dangerous operations.

Fast by design

The first useful suggestion should feel instantaneous: sub-300 ms for locally resolved intents and a fast streamed response for model-assisted requests. A warm interactive session reuses repository context. Exact and semantic caches, local routing, parallel read-only inspection, compact prompts, and constrained outputs protect latency.

gitit combines the request with safe repository facts - current branch, remotes, working-tree state, branches, tags, upstreams, and installed tools such as the GitHub CLI. Context improves accuracy without exposing file contents or mutating the repository.

A phased path to Git intelligence

Useful immediately; private and local over time

Phase 1 - Hosted model, guarded execution

The first release assumes internet access and an available hosted model while remaining deterministic wherever possible. A curated local resolver handles frequent intents and extracts entities such as branches and remotes. A model handles unusual phrasing, ambiguity, and multi-step workflows.

Natural-language request	Repository context	Resolver + model	Policy validator	Confirm / run

The model never returns an arbitrary shell string. It emits a constrained plan: intent, executable, argument list, explanation, confidence, risk, prerequisites, and alternatives. A policy layer validates supported Git and GitHub CLI operations, checks repository facts, rejects unsafe syntax, and selects the required confirmation. Commands use direct process invocation rather than a shell.

Initial command surface

- Status, history, show, diff, blame, and repository inspection
- Branch creation, switching, tracking, renaming, and deletion
- Add, restore, commit, amend, stash, and unstage
- Fetch, pull, push, upstreams, and remotes
- Merge, rebase, cherry-pick, revert, and conflict guidance
- Tags, releases, and pull requests through gh

Safety boundary

Risk is classified before execution. Viewing history is low risk; rewriting commits, force-pushing, deleting branches, or discarding work triggers stronger warnings and explicit confirmation. Structured arguments prevent command chaining and unsafe interpolation. Secrets and sensitive file contents stay out of model context by default.

Latency	Acceptance	Safety	Local share
P95 time to first useful suggestion	Top-one suggestion accepted	No unconfirmed mutation	Requests resolved without network

Phase 2 - A local, open-weight Git specialist

The second phase removes dependence on continuous internet access and general-purpose hosted models. gitit adopts a compact open-weight model selected for intent accuracy, structured-output reliability, latency, memory footprint, license compatibility, and performance across common developer hardware.

Training data pairs diverse requests with validated, platform-aware Git plans. It includes paraphrases, shorthand, misspellings, ambiguity, repository-state fixtures, risk labels, explanations, clarification questions, and unsafe counterexamples. Synthetic examples expand coverage, while Git documentation and disposable-repository tests provide ground truth. Opt-in, privacy-preserving feedback improves real-world language coverage.

The practical path is supervised fine-tuning of an existing open-weight base, followed by preference optimization for candidate ranking and concise explanations - not training a foundation model from scratch. The deterministic resolver remains the fastest path for common commands; the model specializes in ambiguity and workflow composition. The same schema, policy validator, and execution boundary apply in both phases.

Evaluation is behavioral: create repositories in known states, ask thousands of intent variants, execute approved plans in sandboxes, and verify resulting commits, branches, files, remotes, and exit codes. Regression suites emphasize destructive edge cases so that lower latency never weakens safety.

The vision

gitit is not a replacement for Git; it is a more humane way to reach Git. Beginners gain confidence by seeing and understanding every command. Experienced developers recover obscure workflows without context switching. Over time, gitit can explain repository state, guide conflict resolution, compose reviewable multi-step plans, and teach Git incidentally - while preserving one promise: **say what you mean, see what will happen, and stay in control.**